

Inteligencia Artificial y Mini-Robots

Taller 5

Presentado por:

Cristian Camilo García Gamboa - crigarciaga@unal.edu.co

Mateo Becerra Porras - mabecerrap@unal.edu.co

Santiago Ospina Contreras - saospinaco@unal.edu.co

Profesor:

José J. Martínez P.

jmcursosingenieria111@gmail.com

Fecha de entrega:

29 de Septiembre de 2026



Universidad Nacional de Colombia - Sede Bogotá
Facultad de Ingeniería
2026 II

1. Con base en el programa Clase_NNA3, desarrolle dos redes neuronales con 2 capas oculta, una que aprenda a reconocer una compuerta NAND y la otra aprenda a reconocer una compuerta XOR.

Este programa implementa una red neuronal artificial multicapa (MLP) utilizando Python y la librería NumPy. El objetivo de la red es aprender el comportamiento de las compuertas lógicas NAND y XOR mediante un proceso de entrenamiento supervisado. Para ello, la red recibe ejemplos de entrada junto con sus respectivas salidas esperadas y ajusta internamente sus parámetros hasta producir respuestas cada vez más cercanas a las correctas.

Python

```
import numpy as np

np.random.seed(1)

# -----
# Funcion de activacion sigmoide y su derivada
# -----
def sig(z):
    return 1 / (1 + np.exp(-z))

def dsig(a):
    # Propiedad usada en el documento: sigma' = sigma * (1 - sigma)
    # 'a' ya es la salida de sig(z), por eso no se vuelve a aplicar sig()
    return a * (1 - a)

# -----
# Clase: Red neuronal feed-forward con 2 capas ocultas
# -----
class RedNeuronal:
    def __init__(self, n0, n1, n2, n3, eta=0.5):
        """
        n0: numero de entradas (features)
        n1: neuronas en la capa oculta 1
        n2: neuronas en la capa oculta 2
        n3: neuronas en la capa de salida
        eta: tasa de aprendizaje
        """
        self.eta = eta
        # Inicializacion aleatoria de pesos y sesgos, valores entre -1 y 1
        self.W1 = np.random.uniform(-1, 1, (n1, n0))
        self.b1 = np.random.uniform(-1, 1, (n1, 1))
        self.W2 = np.random.uniform(-1, 1, (n2, n1))
        self.b2 = np.random.uniform(-1, 1, (n2, 1))
```

```

self.W3 = np.random.uniform(-1, 1, (n3, n2))
self.b3 = np.random.uniform(-1, 1, (n3, 1))

def propaga(self, X):
    """Propagacion hacia adelante. X: vector columna (n0,1)."""
    Z1 = np.dot(self.W1, X) + self.b1
    A1 = sig(Z1)
    Z2 = np.dot(self.W2, A1) + self.b2
    A2 = sig(Z2)
    Z3 = np.dot(self.W3, A2) + self.b3
    A3 = sig(Z3)
    return A3, A2, A1

def backpropagation(self, X, Yr, A3, A2, A1):
    """Calcula el error, lo retropropaga capa por capa y ajusta pesos."""
    e = Yr - A3 # error de clasificacion en la salida

    # ---- Capa de salida (3) ----
    delta3 = e * dsig(A3)
    dW3 = np.dot(delta3, A2.T)
    db3 = delta3

    # ---- Capa oculta 2 ----
    delta2 = np.dot(self.W3.T, delta3) * dsig(A2)
    dW2 = np.dot(delta2, A1.T)
    db2 = delta2

    # ---- Capa oculta 1 ----
    delta1 = np.dot(self.W2.T, delta2) * dsig(A1)
    dW1 = np.dot(delta1, X.T)
    db1 = delta1

    # ---- Actualizacion de pesos y sesgos (descenso de gradiente) ----
    self.W3 += self.eta * dW3
    self.b3 += self.eta * db3
    self.W2 += self.eta * dW2
    self.b2 += self.eta * db2
    self.W1 += self.eta * dW1
    self.b1 += self.eta * db1

    return np.mean(np.abs(e))

def entrenar(self, X_set, Y_set, epocas=20000, verbose=True):
    """Entrena la red con todo el conjunto de entrenamiento, epoca por epoca."""
    errores = []
    for epoca in range(epocas):

```

```

        error_epoca = 0.0
        for X, Yr in zip(X_set, Y_set):
            X = X.reshape(-1, 1)
            Yr = Yr.reshape(-1, 1)
            A3, A2, A1 = self.propaga(X)
            error_epoca += self.backpropagation(X, Yr, A3, A2, A1)
        error_promedio = error_epoca / len(X_set)
        errores.append(error_promedio)
        if verbose and epoca % 2000 == 0:
            print(f" Epoca {epoca:6d} Error promedio: {error_promedio:.6f}")
    return errores

def predecir(self, X):
    X = X.reshape(-1, 1)
    A3, _, _ = self.propaga(X)
    return A3

# -----
# Programa principal: entrenamiento de NAND y XOR
# -----
if __name__ == "__main__":
    entradas = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])

    # ===== Compuerta NAND =====
    print("=" * 55)
    print("Entrenando red neuronal para la compuerta NAND")
    print("=" * 55)
    salidas_nand = np.array([[1], [1], [1], [0]])

    red_nand = RedNeuronal(n0=2, n1=4, n2=4, n3=1, eta=0.5)
    red_nand.entrenar(entradas, salidas_nand, epocas=20000)

    print("\nResultados NAND:")
    for x, y_esperado in zip(entradas, salidas_nand):
        y = red_nand.predecir(x)
        print(f" Entrada: {x} -> Salida red: {y[0,0]:.4f} "
              f"~ {round(y[0,0])} (esperado: {y_esperado[0]})")

    # ===== Compuerta XOR =====
    print("\n" + "=" * 55)
    print("Entrenando red neuronal para la compuerta XOR")
    print("=" * 55)
    salidas_xor = np.array([[0], [1], [1], [0]])

    red_xor = RedNeuronal(n0=2, n1=4, n2=4, n3=1, eta=0.5)
    red_xor.entrenar(entradas, salidas_xor, epocas=20000)

```

```
print("\nResultados XOR:")
for x, y_esperado in zip(entradas, salidas_xor):
    y = red_xor.predecir(x)
    print(f"  Entrada: {x}  ->  Salida red: {y[0,0]:.4f}  "
          f"~ {round(y[0,0])}  (esperado: {y_esperado[0]})")
```

Inicialmente se importa la librería NumPy, que permite trabajar eficientemente con vectores y matrices, estructuras fundamentales en el funcionamiento de las redes neuronales. También se establece una semilla aleatoria para garantizar que la inicialización de los pesos sea siempre la misma en cada ejecución, permitiendo reproducir los resultados obtenidos.

A continuación, se define la función de activación sigmoide y su derivada. La función sigmoide transforma cualquier valor numérico en un valor comprendido entre 0 y 1, permitiendo interpretar la salida de las neuronas como un nivel de activación. La derivada de esta función es utilizada durante el entrenamiento para determinar cómo deben modificarse los pesos de la red con el fin de reducir el error de predicción.

La estructura principal se encuentra en la clase RedNeuronal, que representa una red neuronal feed-forward compuesta por una capa de entrada, dos capas ocultas y una capa de salida. Al crear la red se especifica el número de neuronas de cada capa y la tasa de aprendizaje. Además, se inicializan de manera aleatoria las matrices de pesos y los vectores de sesgos, que son los parámetros encargados de almacenar el conocimiento adquirido durante el entrenamiento.

El método `propaga()` implementa la propagación hacia adelante. En este proceso, la información fluye desde las entradas hacia la salida pasando por cada una de las capas ocultas. Cada neurona recibe los valores de la capa anterior, calcula una combinación ponderada utilizando sus pesos y sesgos, y posteriormente aplica la función sigmoide para generar su activación. El resultado obtenido en una capa se utiliza como entrada de la siguiente hasta llegar a la capa de salida, donde se obtiene la predicción final de la red.

El aprendizaje de la red se realiza mediante el método `backpropagation()`. Primero se calcula el error comparando la salida obtenida con la salida esperada. Posteriormente, dicho error se propaga en sentido inverso a través de las capas de la red para determinar qué tan responsable fue cada neurona del error cometido. A partir de esta información se calculan los ajustes necesarios en los pesos y sesgos de cada capa. Finalmente, estos parámetros son actualizados utilizando la tasa de aprendizaje establecida, permitiendo que la red mejore progresivamente sus predicciones.

El método `entrenar()` coordina todo el proceso de aprendizaje. Durante cada época, la red procesa todos los ejemplos del conjunto de entrenamiento. Para cada patrón realiza una propagación hacia adelante, calcula el error correspondiente y ejecuta la retropropagación

para actualizar los pesos. Este ciclo se repite miles de veces hasta que el error promedio disminuye y la red logra representar adecuadamente la relación entre las entradas y las salidas esperadas.

Una vez finalizado el entrenamiento, el método `predecir()` permite utilizar la red para realizar nuevas clasificaciones. Este método recibe una entrada, la propaga a través de la red utilizando los pesos aprendidos y devuelve la salida calculada, sin modificar los parámetros internos.

En la sección principal del programa se construye el conjunto de entrenamiento utilizando las cuatro combinaciones posibles de dos variables binarias. Primero se entrena una red para aprender la compuerta NAND, cuya salida es 1 en todos los casos excepto cuando ambas entradas son 1. Posteriormente se crea y entrena una segunda red para aprender la compuerta XOR, cuya salida es 1 únicamente cuando las entradas son diferentes.

La elección de la compuerta XOR resulta especialmente interesante porque representa un problema más complejo que no puede resolverse mediante una única neurona. Gracias a la utilización de varias capas ocultas, la red es capaz de aprender relaciones no lineales entre las entradas y las salidas, demostrando la capacidad de las redes neuronales multicapa para resolver problemas de clasificación más complejos.

En términos generales, el programa muestra el funcionamiento completo de una red neuronal artificial: inicialización de pesos, propagación de la información, cálculo del error, retropropagación y actualización de parámetros. A través de este proceso iterativo de aprendizaje, la red logra identificar patrones y reproducir correctamente el comportamiento lógico de las compuertas NAND y XOR.

Explicación de Resultados:

Los resultados obtenidos muestran que la red neuronal logró aprender correctamente el comportamiento de las compuertas lógicas NAND y XOR. Esto puede observarse tanto en la disminución progresiva del error durante el entrenamiento como en la precisión de las salidas generadas al finalizar el proceso.

```
=====
Entrenando red neuronal para la compuerta NAND
=====
Epoca      0  Error promedio: 0.411340
Epoca   2000  Error promedio: 0.015086
Epoca   4000  Error promedio: 0.009609
Epoca   6000  Error promedio: 0.007515
Epoca   8000  Error promedio: 0.006346
Epoca  10000  Error promedio: 0.005580
Epoca  12000  Error promedio: 0.005029
Epoca  14000  Error promedio: 0.004609
Epoca  16000  Error promedio: 0.004277
Epoca  18000  Error promedio: 0.004005
```

Para la compuerta NAND, el error promedio inicial fue de aproximadamente 0.411, lo que indica que al comienzo la red realizaba predicciones bastante alejadas de los valores esperados debido a que los pesos habían sido inicializados aleatoriamente. Sin embargo, a medida que avanzaron las épocas de entrenamiento, el error disminuyó de manera constante hasta alcanzar un valor cercano a 0.004 al final del proceso. Este comportamiento evidencia que el algoritmo de retropropagación logró ajustar progresivamente los pesos y sesgos de la red, permitiéndole aprender la relación existente entre las entradas y las salidas deseadas.

Resultados NAND:

```
Entrada: [0 0] -> Salida red: 0.9996 ~ 1 (esperado: 1)
Entrada: [0 1] -> Salida red: 0.9957 ~ 1 (esperado: 1)
Entrada: [1 0] -> Salida red: 0.9959 ~ 1 (esperado: 1)
Entrada: [1 1] -> Salida red: 0.0063 ~ 0 (esperado: 0)
```

Al analizar las predicciones finales para NAND, se observa que las combinaciones de entrada [0,0], [0,1] y [1,0] producen salidas cercanas a 1, mientras que la combinación [1,1] genera una salida cercana a 0. Aunque los valores obtenidos no son exactamente 0 o 1 debido a la naturaleza continua de la función sigmoide, al aplicar un redondeo todas las predicciones coinciden con los valores esperados de la tabla de verdad. Esto demuestra que la red aprendió correctamente el comportamiento de la compuerta.

```
=====
Entrenando red neuronal para la compuerta XOR
=====
Epoca      0  Error promedio: 0.516136
Epoca   2000  Error promedio: 0.030135
Epoca   4000  Error promedio: 0.014950
Epoca   6000  Error promedio: 0.011051
Epoca   8000  Error promedio: 0.009102
Epoca  10000  Error promedio: 0.007891
Epoca  12000  Error promedio: 0.007049
Epoca  14000  Error promedio: 0.006421
Epoca  16000  Error promedio: 0.005930
Epoca  18000  Error promedio: 0.005533
```

Para la compuerta XOR, el entrenamiento inició con un error promedio ligeramente superior, cercano a 0.516. Este resultado era esperado, ya que XOR representa un problema más complejo debido a que la relación entre las entradas y la salida no es lineal. A pesar de esta dificultad, el error disminuyó progresivamente durante las 20.000 épocas de entrenamiento hasta alcanzar aproximadamente 0.0055, indicando que la red fue capaz de encontrar una representación adecuada del problema mediante las capas ocultas.

Resultados XOR:

```
Entrada: [0 0] -> Salida red: 0.0040 ~ 0 (esperado: 0)
Entrada: [0 1] -> Salida red: 0.9941 ~ 1 (esperado: 1)
Entrada: [1 0] -> Salida red: 0.9948 ~ 1 (esperado: 1)
Entrada: [1 1] -> Salida red: 0.0058 ~ 0 (esperado: 0)
```

Las predicciones finales para XOR muestran un desempeño igualmente satisfactorio. Las entradas [0, 0] y [1, 1] generan salidas muy cercanas a 0, mientras que las entradas [0, 1] y [1, 0] producen valores muy próximos a 1. Nuevamente, al redondear las salidas obtenidas, todas coinciden con los resultados esperados de la compuerta XOR. Esto confirma que la red neuronal logró aprender correctamente una función lógica que no puede ser resuelta mediante una sola neurona.

Otro aspecto importante es la evolución del error durante el entrenamiento. En ambos casos se observa una reducción rápida del error durante las primeras épocas y una disminución más gradual en las etapas finales. Este comportamiento es característico de los algoritmos de aprendizaje basados en descenso por gradiente, donde los ajustes iniciales producen mejoras significativas y posteriormente las modificaciones se vuelven cada vez más pequeñas a medida que la red se aproxima a una solución óptima.

En conclusión, los resultados demuestran que la arquitectura propuesta, compuesta por dos capas ocultas de cuatro neuronas cada una y entrenada mediante retropropagación, fue suficiente para aprender exitosamente tanto la compuerta NAND como la compuerta XOR. La reducción sostenida del error y la coincidencia entre las salidas predichas y las salidas esperadas evidencian que el modelo convergió correctamente y adquirió la capacidad de generalizar el comportamiento lógico de ambas compuertas.

2. Con base en la librería tensorflow, descargue el data set fashion MNIST. Haga una clasificación de prendas de vestir. Explique cada una de las funciones y las principales instrucciones, saque conclusiones.

El código desarrollado implementa una red neuronal artificial utilizando TensorFlow y Keras con el objetivo de clasificar imágenes de prendas de vestir pertenecientes al conjunto de datos Fashion MNIST. Este conjunto de datos contiene miles de imágenes en escala de grises de diferentes categorías de ropa, como camisetas, pantalones, vestidos, zapatos y bolsos. El programa realiza todo el proceso necesario para construir un modelo de clasificación, comenzando con la carga y preparación de los datos, seguido por la creación de la arquitectura de la red neuronal, su entrenamiento mediante aprendizaje supervisado, la evaluación de su desempeño y, finalmente, la visualización de los resultados obtenidos mediante gráficas e imágenes clasificadas.

Python

```
import numpy as np
import tensorflow as tf
import matplotlib
```

```

import matplotlib.pyplot as plt

# -----
# 1. DESCARGA Y CARGA DEL DATASET
# -----
# tf.keras.datasets.fashion_mnist.load_data()
# Descarga el dataset Fashion MNIST desde los servidores de Keras y lo entrega
# ya separado en conjunto de entrenamiento y de prueba.
# Devuelve dos tuplas: (x_train, y_train) y (x_test, y_test)
#   x_train: 60.000 imagenes de 28x28 pixeles (escala de grises, 0-255)
#   y_train: 60.000 rotulos (numeros de 0 a 9, uno por clase de prenda)
#   x_test / y_test: 10.000 imagenes/rotulos para evaluar la red
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.fashion_mnist.load_data()

print("Forma x_train:", x_train.shape)  # (60000, 28, 28)
print("Forma y_train:", y_train.shape)  # (60000,)
print("Forma x_test :", x_test.shape)   # (10000, 28, 28)

# Fashion MNIST tiene 10 categorias de prendas, numeradas de 0 a 9.
# Esta lista muestra el nombre en lugar del numero.
clases = ["Camiseta/top", "Pantalon", "Sueter", "Vestido", "Abrigo",
          "Sandalia", "Camisa", "Zapatilla", "Bolso", "Botin"]

# -----
# 2. PREPROCESAMIENTO
# -----
# Los pixeles vienen en el rango [0, 255]. Los dividimos entre 255.0
# para normalizarlos al rango [0, 1]. Esto es equivalente al vector
# de "caracteristicas" (features) X del documento, pero aqui cada
# imagen aporta 784 caracteristicas (28 x 28 pixeles).
# Normalizar ayuda a que la red converja mas rapido y de forma mas
# estable (evita que la funcion de activacion sigmoide/ReLU trabaje
# con valores muy grandes).
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

# -----
# 3. DEFINICION DE LA RED NEURONAL (topologia)
# -----
# tf.keras.Sequential crea una red "feedforward" (como las vistas en
# el documento): los datos fluyen de una capa a la siguiente, sin
# retroalimentacion.
#

```

```

# Input(shape=(28,28)) -> capa de entrada, cada imagen es una
#                           matriz de 28x28 (no un vector todavia)
# Flatten()                -> convierte la matriz 28x28 en un vector
#                           de 784 valores (el vector X de entrada
#                           de una neurona artificial, ver 5.2.2)
# Dense(128, relu)         -> primera capa oculta (capa 1), 128
#                           neuronas, cada una calcula
#                            $Z = W.X + b$  y aplica la funcion de
#                           activacion ReLU (figura 5.7 del doc.)
# Dense(64, relu)          -> segunda capa oculta (capa 2), 64
#                           neuronas, misma logica que la anterior
# Dense(10, softmax)       -> capa de salida con 10 neuronas (una por
#                           clase de prenda). softmax convierte las
#                           salidas en probabilidades que suman 1,
#                           es la generalizacion multi-clase de la
#                           funcion sigmoide vista en 5.2.2

model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(28, 28)),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(128, activation="relu"),
    tf.keras.layers.Dense(64, activation="relu"),
    tf.keras.layers.Dense(10, activation="softmax"),
])

model.summary()

# model.summary() imprime la topologia de la red: cada capa, su forma
# de salida y el numero de parametros (pesos W y sesgos b) que debe
# aprender. Es el equivalente a las "matrices de pesos" W1, W2, W3
# del documento, pero calculadas automaticamente segun el tamano de
# cada capa.

# -----
# 4. COMPILACION DEL MODELO
# -----
# model.compile() define como se va a entrenar la red:
# optimizer="adam" -> algoritmo de descenso de gradiente que
#                     ademas ajusta automaticamente la tasa de
#                     aprendizaje (eta en el documento) durante
#                     el entrenamiento. Es una version mejorada
#                     del descenso de gradiente simple visto en
#                     la seccion de Backpropagation.
# loss="sparse_categorical_crossentropy"
#                     -> funcion de perdida para clasificacion
#                     multi-clase cuando los rotulos son
#                     numeros enteros (0 a 9) y no vectores

```

```

#             one-hot. Es analoga a la funcion de
#             perdida E = 1/2 (e)^2 del documento, pero
#             adaptada a probabilidades (softmax).
# metrics=["accuracy"] -> ademas de la perdida, se reporta el
#             porcentaje de aciertos en cada epoca.
model.compile(optimizer="adam",
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])

# -----
# 5. ENTRENAMIENTO
# -----
# model.fit() ejecuta el proceso de aprendizaje supervisado: por cada
# epoca, TensorFlow propaga las imagenes hacia adelante, calcula el
# error contra los rotulos verdaderos y aplica Backpropagation para
# ajustar todos los pesos W y sesgos b de la red (el mismo algoritmo
# explicado en la seccion 5.5 del documento, pero implementado de
# forma optimizada internamente).
# epochs=10          -> numero de veces que la red ve todo el
#                     conjunto de entrenamiento completo
# batch_size=64      -> en vez de actualizar los pesos ejemplo por
#                     ejemplo, se agrupan 64 imagenes y se
#                     actualiza el promedio del error de ese
#                     grupo (mas rapido y estable)
# validation_split=0.1 -> aparta el 10% del entrenamiento como
#                     conjunto de validacion, para revisar que
#                     la red generaliza y no solo memoriza
#                     (ver "Generalizacion del aprendizaje",
#                     seccion 5.4 del documento)
historial = model.fit(x_train, y_train,
                    epochs=10,
                    batch_size=64,
                    validation_split=0.1,
                    verbose=2)

# -----
# 6. EVALUACION SOBRE EL CONJUNTO DE PRUEBA
# -----
# model.evaluate() propaga el conjunto de prueba (que la red nunca
# vio durante el entrenamiento) y devuelve la perdida final y la
# exactitud (accuracy) real del modelo. Este es el "conjunto de
# prueba" que menciona el documento para estimar el comportamiento
# de la red en el mundo real.
perdida_test, exactitud_test = model.evaluate(x_test, y_test, verbose=0)
print(f"\nPerdida en el conjunto de prueba: {perdida_test:.4f}")
print(f"Exactitud en el conjunto de prueba: {exactitud_test:.4f} ")

```

```

f"({exactitud_test*100:.2f} %)")

# -----
# 7. PREDICCIONES DE EJEMPLO
# -----
# model.predict() propaga nuevas imagenes a traves de la red ya
# entrenada y devuelve, para cada una, el vector de 10 probabilidades
# (una por clase). np.argmax() selecciona la clase con mayor
# probabilidad, es decir, la prediccion final de la red.

probs = model.predict(x_test[:9], verbose=0)
predicciones = np.argmax(probs, axis=1)

print("\nEjemplos de prediccion (Prediccion vs Real):")
for i in range(9):
    print(f" {clases[predicciones[i]]:<15} vs {clases[y_test[i]]}")

# -----
# 8. GRAFICAS
# -----
# Se grafican la perdida y la exactitud del entrenamiento y de la
# validacion en cada epoca. Esto permite observar visualmente si la
# red esta en underfitting, en un ajuste adecuado, o en overfitting
# (ver figura 5.12 del documento).
# -----
# 8. GRAFICAS
# -----

# Curvas de entrenamiento
fig, axs = plt.subplots(1, 2, figsize=(11, 4))

axs[0].plot(historial.history["loss"], label="Entrenamiento")
axs[0].plot(historial.history["val_loss"], label="Validacion")
axs[0].set_title("Evolucion de la funcion de perdida")
axs[0].set_xlabel("Epoca")
axs[0].set_ylabel("Perdida")
axs[0].legend()

axs[1].plot(historial.history["accuracy"], label="Entrenamiento")
axs[1].plot(historial.history["val_accuracy"], label="Validacion")
axs[1].set_title("Evolucion de la exactitud")
axs[1].set_xlabel("Epoca")
axs[1].set_ylabel("Exactitud")
axs[1].legend()

plt.tight_layout()

```

```

# Mostrar en pantalla
plt.show()

# Guardar opcionalmente
plt.savefig("curvas_entrenamiento.png")

# -----
# Ejemplos de prediccion
# -----

fig2, axs2 = plt.subplots(3, 3, figsize=(7, 7))

for i, ax in enumerate(axs2.flat):
    ax.imshow(x_test[i], cmap="gray")

    correcto = "OK" if predicciones[i] == y_test[i] else "X"

    ax.set_title(
        f"P: {clases[predicciones[i]]}\n"
        f"R: {clases[y_test[i]]} [{correcto}]",
        fontsize=8
    )

    ax.axis("off")

plt.tight_layout()

# Mostrar en pantalla
plt.show()

# Guardar opcionalmente
plt.savefig("ejemplos_prediccion.png")

# -----
# 9. CONCLUSIONES (generadas a partir de los resultados obtenidos)
# -----

acc_train_final = historial.history["accuracy"][-1]
acc_val_final = historial.history["val_accuracy"][-1]
brecha = acc_train_final - acc_val_final

print("\n" + "=" * 60)
print("CONCLUSIONES")
print("=" * 60)
print(f"- Exactitud final en entrenamiento: {acc_train_final*100:.2f} %")

```

```

print(f"- Exactitud final en validacion:      {acc_val_final*100:.2f} %")
print(f"- Exactitud en el conjunto de prueba: {exactitud_test*100:.2f} %")

if brecha > 0.05:
    print("- La exactitud de entrenamiento es notablemente mayor que la de "
          "validacion, lo que sugiere un principio de sobre-entrenamiento "
          "(overfitting): la red esta memorizando detalles del conjunto "
          "de entrenamiento en vez de generalizar. Se recomendaria reducir "
          "el numero de epocas, agregar regularizacion (Dropout) o "
          "aumentar los datos de entrenamiento.")
else:
    print("- La exactitud de entrenamiento y validacion son similares, lo "
          "que indica que la red esta generalizando adecuadamente y no "
          "presenta un sobre-entrenamiento marcado.")

```

El primer paso del programa consiste en cargar el conjunto de datos Fashion MNIST. Para ello se utiliza una función incorporada en TensorFlow que descarga automáticamente las imágenes y las divide en dos grupos: un conjunto de entrenamiento utilizado para que la red aprenda los patrones presentes en las imágenes y un conjunto de prueba empleado posteriormente para evaluar qué tan bien generaliza el modelo frente a datos que no ha visto anteriormente. Una vez cargados los datos, se realiza una etapa de preprocesamiento en la que los valores de los píxeles son convertidos a formato decimal y normalizados a un rango entre 0 y 1. Este procedimiento facilita el aprendizaje de la red y contribuye a mejorar la estabilidad numérica durante el entrenamiento.

Posteriormente se construye la arquitectura de la red neuronal utilizando un modelo secuencial. La primera capa recibe imágenes de 28×28 píxeles y posteriormente una capa de aplanamiento transforma cada imagen bidimensional en un vector unidimensional que puede ser procesado por las capas densas. A continuación, se incorporan dos capas ocultas completamente conectadas que contienen 128 y 64 neuronas respectivamente. Estas capas son las encargadas de identificar patrones relevantes dentro de las imágenes, tales como formas, bordes y combinaciones de características que permiten diferenciar unas prendas de otras. Finalmente, la capa de salida está compuesta por diez neuronas, una para cada categoría disponible en el conjunto de datos, y utiliza una función de activación que permite convertir las salidas en probabilidades asociadas a cada clase.

Una vez definida la arquitectura, el modelo es configurado especificando los elementos necesarios para el aprendizaje. Se emplea un optimizador encargado de actualizar automáticamente los pesos de la red con el objetivo de reducir el error, una función de pérdida que cuantifica las diferencias entre las predicciones y las respuestas correctas, y una métrica de exactitud que permite medir el porcentaje de clasificaciones correctas obtenidas durante el entrenamiento. Además, se genera un resumen de la red que muestra la estructura

completa del modelo y el número de parámetros que serán ajustados durante el proceso de aprendizaje.

El entrenamiento de la red neuronal se realiza mediante un procedimiento iterativo en el que las imágenes son procesadas en pequeños grupos denominados lotes. Durante cada época, la información es propagada desde la entrada hasta la salida para generar una predicción, posteriormente se calcula el error cometido y este es utilizado para ajustar los pesos mediante el algoritmo de retropropagación. Este proceso se repite múltiples veces hasta que la red logra minimizar progresivamente el error y mejorar su capacidad de clasificación. Paralelamente, una parte de los datos de entrenamiento es reservada para validación, permitiendo monitorear el desempeño del modelo durante el aprendizaje y detectar posibles problemas de sobreajuste.

Una vez finalizado el entrenamiento, el modelo es evaluado utilizando el conjunto de prueba. En esta etapa se calcula la pérdida final y la exactitud alcanzada por la red utilizando imágenes que no participaron en el proceso de aprendizaje. Esto permite obtener una medida objetiva del desempeño real del modelo y verificar su capacidad para generalizar correctamente a nuevos ejemplos.

Posteriormente se realizan predicciones individuales sobre las imágenes de prueba. Para cada imagen, la red calcula la probabilidad de pertenencia a cada una de las diez categorías disponibles y selecciona aquella que presenta la mayor probabilidad. De esta manera se obtiene la clasificación final generada por el modelo, la cual puede compararse con la etiqueta real de la imagen para verificar si la predicción fue correcta.

Finalmente, el programa genera diferentes representaciones gráficas para facilitar el análisis de los resultados. Se construyen curvas que muestran la evolución de la función de pérdida y de la exactitud tanto para el conjunto de entrenamiento como para el de validación, permitiendo observar cómo mejora el desempeño de la red a medida que avanzan las épocas. Adicionalmente, se presentan varias imágenes del conjunto de prueba junto con sus respectivas predicciones y etiquetas reales, lo que proporciona una visualización directa de la capacidad de clasificación alcanzada por el modelo. Estas gráficas permiten evaluar de manera intuitiva el comportamiento de la red neuronal y constituyen una herramienta útil para interpretar los resultados obtenidos durante el experimento.

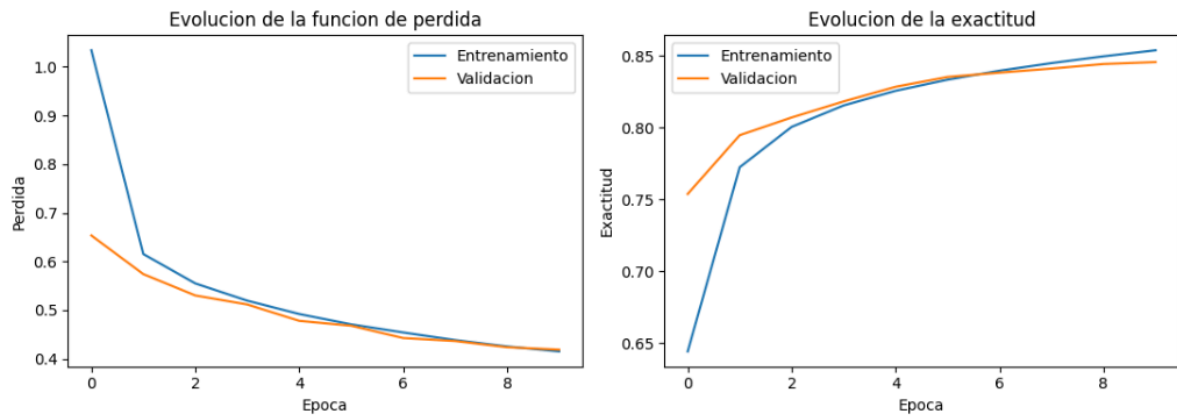
Explicación de los resultados obtenidos

Los resultados evidencian que la red neuronal aprendió correctamente a clasificar las prendas del conjunto de datos Fashion MNIST. Esto se observa en la disminución progresiva de la función de pérdida y en el aumento constante de la exactitud tanto en entrenamiento como en validación a medida que avanzan las épocas.

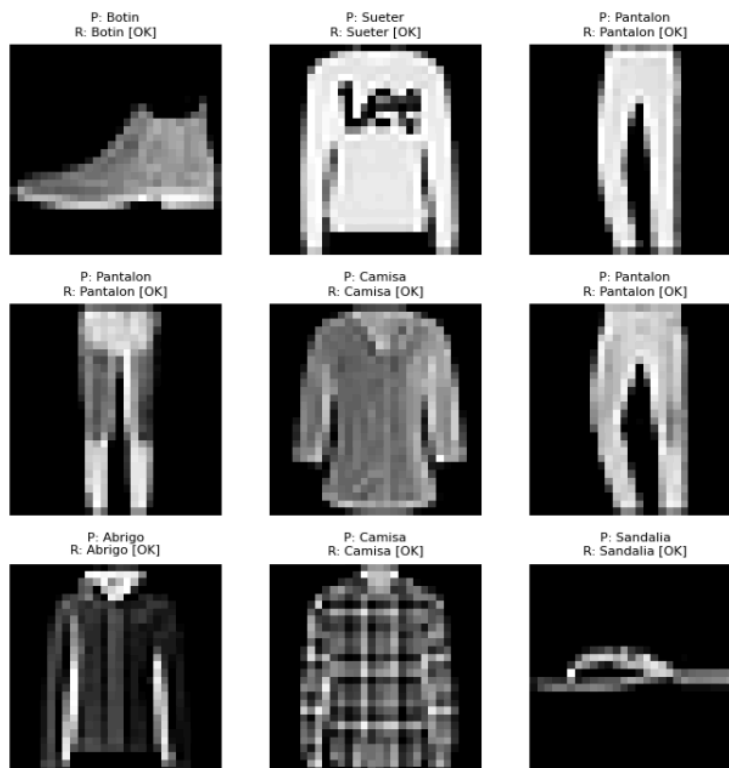
Perdida en el conjunto de prueba: 0.4502
Exactitud en el conjunto de prueba: 0.8387 (83.87 %)

Ejemplos de prediccion (Prediccion vs Real):

Botin	vs	Botin
Sueter	vs	Sueter
Pantalon	vs	Pantalon
Pantalon	vs	Pantalon
Camisa	vs	Camisa
Pantalon	vs	Pantalon
Abrigo	vs	Abrigo
Camisa	vs	Camisa
Sandalia	vs	Sandalia



Las curvas de entrenamiento y validación presentan un comportamiento muy similar, lo que indica que el modelo generaliza adecuadamente y no muestra señales importantes de sobreajuste. Al finalizar el proceso, se obtuvo una exactitud del 85.37 % en entrenamiento, 84.55 % en validación y 83.87 % en el conjunto de prueba, valores bastante cercanos entre sí.



Las imágenes de ejemplo confirman este desempeño, ya que todas las prendas mostradas fueron clasificadas correctamente. La red logró identificar categorías como botines,

pantalones, camisas, abrigos, suéteres y sandalias, demostrando que fue capaz de aprender características relevantes de cada clase.

```
=====
CONCLUSIONES
=====
- Exactitud final en entrenamiento: 85.37 %
- Exactitud final en validacion:      84.55 %
- Exactitud en el conjunto de prueba: 83.87 %
```

En general, los resultados indican que la arquitectura implementada fue adecuada para el problema planteado, alcanzando un buen nivel de precisión y una capacidad de generalización satisfactoria sobre datos no vistos durante el entrenamiento.

3. De la librería de Kaggle para Python, baje un dataset, estudie sus características (features) y su rótulo. Genere un modelo y tome ejemplos de internet y analice los resultados.

Para el punto 3 se desarrolló una Red Neuronal Convolutiva (CNN) utilizando TensorFlow y Keras con el conjunto de datos Intel Image Classification. El objetivo del programa es entrenar un modelo capaz de reconocer y clasificar imágenes en diferentes categorías, aplicando los principios de aprendizaje supervisado vistos en el tema de Redes Neuronales Artificiales.

```
Python
import os, random, io
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras import layers, models

!pip -q install kagglehub
import kagglehub

SEED = 42
random.seed(SEED); np.random.seed(SEED); tf.random.set_seed(SEED)
print("TensorFlow:", tf.__version__)
print("GPU:", tf.config.list_physical_devices('GPU'))

path = kagglehub.dataset_download("puneet6060/intel-image-classification")
print("Descargado en:", path)

# Buscar las carpetas que contienen directamente las carpetas de clases
dirs = [dp for dp, dn, _ in os.walk(path) if 'buildings' in dn]
train_dir = [d for d in dirs if 'train' in d.lower()][0]
```

```

test_dir = [d for d in dirs if 'test' in d.lower()][0]
print("Train:", train_dir)
print("Test :", test_dir)

class_names = sorted(os.listdir(train_dir))
print("Clases (rótulos):", class_names, "-> k =", len(class_names))

def contar(d):
    return {c: len(os.listdir(os.path.join(d, c))) for c in class_names}

df = pd.DataFrame({"train": contar(train_dir), "test": contar(test_dir)})
df.loc["TOTAL"] = df.sum()
display(df)

df.drop("TOTAL").plot(kind="bar", figsize=(8,4), title="Imágenes por clase")
plt.ylabel("cantidad"); plt.tight_layout(); plt.show()

# Muestra de imágenes de cada clase
from PIL import Image
fig, axes = plt.subplots(len(class_names), 5, figsize=(12, 14))
sizes = []
for i, c in enumerate(class_names):
    files = random.sample(os.listdir(os.path.join(train_dir, c)), 5)
    for j, f in enumerate(files):
        img = Image.open(os.path.join(train_dir, c, f))
        sizes.append(img.size)
        axes[i, j].imshow(img); axes[i, j].axis("off")
        if j == 0: axes[i, j].set_title(c, loc="left", fontsize=12)
plt.tight_layout(); plt.show()
print("Tamaños de imagen observados (ancho, alto):", set(sizes))

IMG_SIZE = (150, 150)
BATCH = 32

train_ds = tf.keras.utils.image_dataset_from_directory(
    train_dir, validation_split=0.2, subset="training", seed=SEED,
    image_size=IMG_SIZE, batch_size=BATCH)
val_ds = tf.keras.utils.image_dataset_from_directory(
    train_dir, validation_split=0.2, subset="validation", seed=SEED,
    image_size=IMG_SIZE, batch_size=BATCH)
test_ds = tf.keras.utils.image_dataset_from_directory(
    test_dir, image_size=IMG_SIZE, batch_size=BATCH, shuffle=False)

assert train_ds.class_names == class_names
AUTOTUNE = tf.data.AUTOTUNE
train_ds = train_ds.prefetch(AUTOTUNE)

```

```

val_ds = val_ds.prefetch(AUTOTUNE)
test_ds = test_ds.prefetch(AUTOTUNE)

for xb, yb in train_ds.take(1):
    print("Forma de un lote X:", xb.shape, "| rango de píxeles:",
          float(tf.reduce_min(xb)), "-", float(tf.reduce_max(xb)))
    print("Forma de un lote Y:", yb.shape, "| ejemplo de rótulos:", yb[:8].numpy())

data_aug = tf.keras.Sequential([
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.08),
    layers.RandomZoom(0.1),
])

model = models.Sequential([
    layers.Input(shape=IMG_SIZE + (3,)),
    data_aug,
    layers.Rescaling(1./255), # normaliza píxeles a [0,1]
    layers.Conv2D(32, 3, activation="relu"), layers.MaxPooling2D(),
    layers.Conv2D(64, 3, activation="relu"), layers.MaxPooling2D(),
    layers.Conv2D(128, 3, activation="relu"), layers.MaxPooling2D(),
    layers.Flatten(),
    layers.Dropout(0.5),
    layers.Dense(128, activation="relu"),
    layers.Dense(len(class_names), activation="softmax"),
])

model.compile(optimizer="adam",
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])
model.summary()

callbacks = [tf.keras.callbacks.EarlyStopping(monitor="val_loss", patience=4,
                                              restore_best_weights=True)]
history = model.fit(train_ds, validation_data=val_ds, epochs=20, callbacks=callbacks)

# Evolución de la función de pérdida y de la exactitud (como la figura 5.17 del
# documento)
h = history.history
fig, ax = plt.subplots(1, 2, figsize=(12, 4))
ax[0].plot(h["loss"], label="Pérdida de entrenamiento")
ax[0].plot(h["val_loss"], label="Pérdida de validación")
ax[0].set_title("Evolución de la función de pérdida"); ax[0].set_xlabel("Épocas");
ax[0].legend(); ax[0].grid(True)
ax[1].plot(h["accuracy"], label="Exactitud entrenamiento")
ax[1].plot(h["val_accuracy"], label="Exactitud validación")

```

```

ax[1].set_title("Exactitud"); ax[1].set_xlabel("Épocas"); ax[1].legend();
ax[1].grid(True)
plt.tight_layout(); plt.show()

from sklearn.metrics import confusion_matrix, classification_report,
ConfusionMatrixDisplay

test_loss, test_acc = model.evaluate(test_ds, verbose=0)
print(f"Exactitud en prueba: {test_acc:.3f} | Pérdida: {test_loss:.3f}")

y_true = np.concatenate([y.numpy() for _, y in test_ds])
y_pred = np.argmax(model.predict(test_ds, verbose=0), axis=1)

print(classification_report(y_true, y_pred, target_names=class_names, digits=3))

fig, ax = plt.subplots(figsize=(6, 6))
ConfusionMatrixDisplay(confusion_matrix(y_true, y_pred),
display_labels=class_names).plot(
    ax=ax, xticks_rotation=45, colorbar=False)
plt.title("Matriz de confusión (prueba)"); plt.tight_layout(); plt.show()

```

Explicación del código

La primera sección del programa importa las librerías necesarias para el desarrollo del proyecto. Se utilizan NumPy para operaciones matemáticas, Pandas para el manejo de datos, Matplotlib para la generación de gráficos y TensorFlow/Keras para la construcción y entrenamiento de la red neuronal. También se instala y utiliza la librería KaggleHub, que permite descargar automáticamente el conjunto de datos desde Kaggle.

Adicionalmente, se define una semilla aleatoria (SEED = 42) para garantizar que los resultados obtenidos sean reproducibles.

Posteriormente, el programa descarga el conjunto de imágenes denominado Intel Image Classification. Una vez descargado, busca automáticamente las carpetas que contienen los datos de entrenamiento y prueba. Estas carpetas están organizadas por clases que actúan como rótulos (labels) para la red. Cada imagen aporta las características visuales (features) y la carpeta define la salida esperada, siguiendo el esquema de aprendizaje supervisado.

Antes de entrenar el modelo, el código realiza un análisis exploratorio de los datos. Para ello identifica todas las clases disponibles y cuenta la cantidad de imágenes existentes en cada una. Luego se genera una gráfica de barras que permite visualizar la distribución de imágenes por categoría. También se muestran ejemplos aleatorios de imágenes pertenecientes a cada clase. Esta etapa es importante porque permite verificar que los datos están correctamente

organizados y proporciona una primera comprensión de las características visuales que la red deberá aprender.

Una vez revisadas las imágenes, el programa crea tres conjuntos de datos:

- Entrenamiento (80%): utilizado para ajustar los pesos de la red neuronal.
- Validación (20%): utilizado para monitorear el desempeño durante el entrenamiento.
- Prueba: utilizado únicamente para evaluar el modelo una vez finalizado el aprendizaje.

Todas las imágenes son redimensionadas a un tamaño uniforme de 150×150 píxeles, permitiendo que la red procese entradas con dimensiones consistentes. Además, los datos son agrupados en lotes (batches) de 32 imágenes y se optimiza su carga mediante la técnica de prefetching, lo que mejora la velocidad de entrenamiento.

La red neuronal utilizada es una CNN (Convolutional Neural Network) compuesta por capas convolucionales, capas de agrupamiento (pooling), una capa densa y una capa de salida con activación Softmax. Las capas convolucionales extraen automáticamente características relevantes de las imágenes, mientras que las capas finales realizan la clasificación de acuerdo con las categorías definidas. Este funcionamiento sigue el principio de propagación de información a través de diferentes capas de la red neuronal.

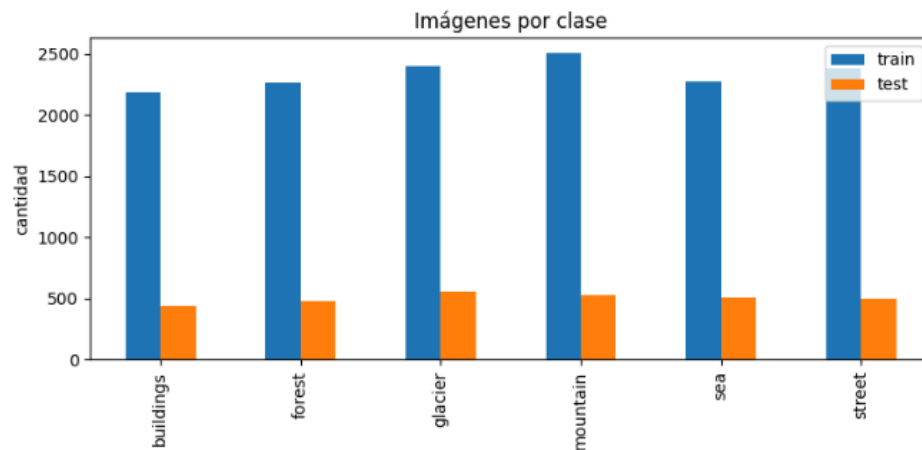
Una vez construida la arquitectura, el modelo se compila utilizando el optimizador Adam y una función de pérdida adecuada para problemas de clasificación multiclase. Durante el entrenamiento, la red compara sus predicciones con los rótulos reales, calcula el error y ajusta sus pesos mediante el algoritmo de retropropagación (Backpropagation), buscando minimizar progresivamente la función de pérdida.

Finalmente, el programa evalúa el desempeño del modelo utilizando imágenes que no participaron en el entrenamiento. Para ello calcula métricas de clasificación, genera una matriz de confusión y muestra gráficas de precisión y pérdida. Estos resultados permiten verificar qué tan bien aprendió la red neuronal y su capacidad para clasificar correctamente nuevas imágenes, cumpliendo el objetivo principal del proyecto.

Explicación de los resultados obtenidos

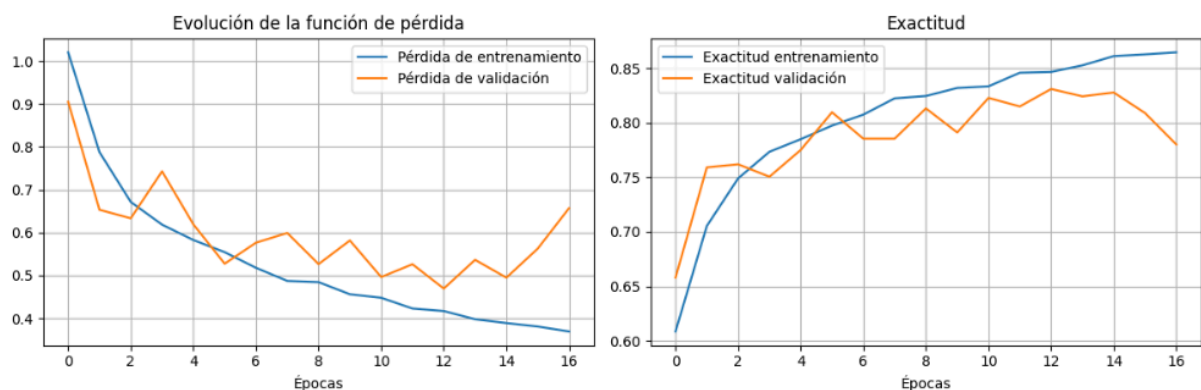
Clases (rótulos): ['buildings', 'forest', 'glacier', 'mountain', 'sea', 'street'] -> k = 6

	train	test
buildings	2191	437
forest	2271	474
glacier	2404	553
mountain	2512	525
sea	2274	510
street	2382	501
TOTAL	14034	3000



La exploración inicial de los datos mostró que el conjunto está compuesto por 6 clases (buildings, forest, glacier, mountain, sea y street), con un total de 14.034 imágenes para entrenamiento y 3.000 para prueba. La distribución de imágenes es bastante equilibrada entre las categorías, lo que favorece un aprendizaje más uniforme de la red neuronal.

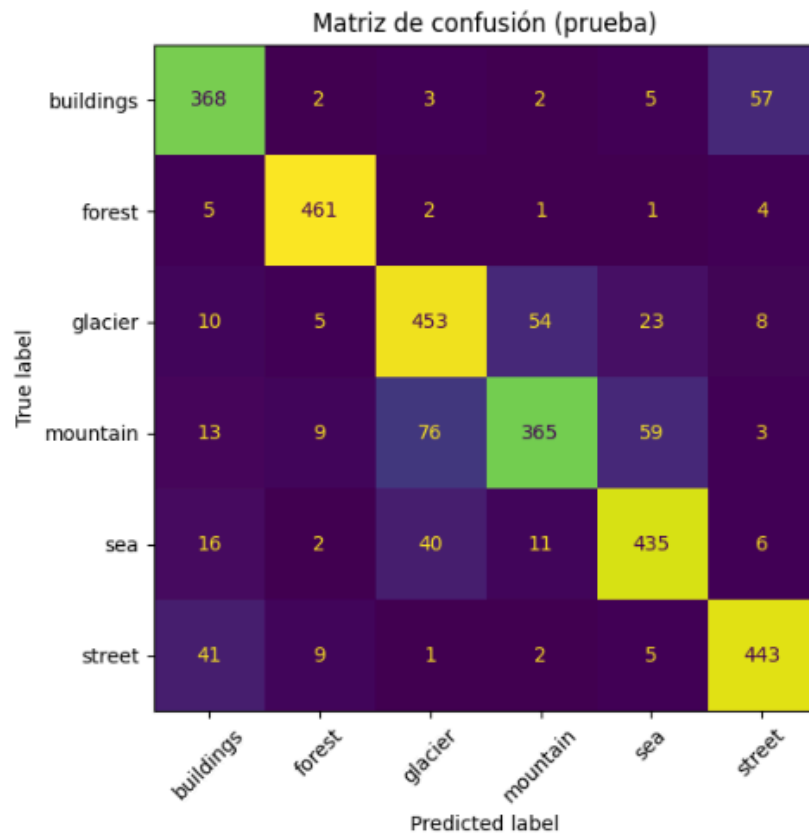
Las imágenes de ejemplo permitieron verificar que cada categoría está correctamente etiquetada y evidenciaron que algunas clases, como mountain, glacier y sea, presentan características visuales similares, lo que puede generar errores de clasificación durante el entrenamiento.



Durante el proceso de entrenamiento, la pérdida disminuyó progresivamente mientras que la exactitud aumentó hasta aproximadamente un 86% en entrenamiento y un 83% en validación. Esto indica que la red aprendió correctamente los patrones presentes en las imágenes, aunque se observa un ligero sobreajuste en las últimas épocas.

Exactitud en prueba: 0.842 Pérdida: 0.468				
	precision	recall	f1-score	support
buildings	0.812	0.842	0.827	437
forest	0.945	0.973	0.958	474
glacier	0.788	0.819	0.803	553
mountain	0.839	0.695	0.760	525
sea	0.824	0.853	0.838	510
street	0.850	0.884	0.867	501
accuracy			0.842	3000
macro avg	0.843	0.844	0.842	3000
weighted avg	0.842	0.842	0.840	3000

La evaluación final arrojó una exactitud de 84.2%, lo que significa que la red clasificó correctamente cerca de 84 de cada 100 imágenes. Entre las categorías evaluadas, forest obtuvo el mejor desempeño (F1-Score = 0.958), mientras que mountain presentó los resultados más bajos (F1-Score = 0.760), debido a la similitud visual con otras categorías.



La matriz de confusión confirma que la mayoría de los errores ocurrieron entre las clases mountain y glacier, así como entre mountain y sea. En general, los resultados muestran que la red neuronal convolucional implementada logró aprender de manera adecuada las características de las imágenes y alcanzar un desempeño satisfactorio en la tarea de clasificación.

4. Punto libre.

Para el punto 4, correspondiente al ejercicio libre, se desarrolló una aplicación de redes neuronales artificiales para predecir si un pedido llegará tarde o a tiempo. El problema se plantea como una clasificación binaria, donde la salida es 1 cuando el pedido llega tarde y 0 cuando llega a tiempo. Para realizar la predicción se utilizan cuatro características: distancia del envío, peso del pedido, presencia de lluvia y salida en hora pico. El ejercicio se relaciona directamente con el aprendizaje supervisado explicado en la clase, en el que la red aprende a partir de datos de entrada asociados con un rótulo o resultado esperado.

Python

```
import numpy as np, pandas as pd, matplotlib.pyplot as plt
np.random.seed(7)

m = 3000 # número de ejemplos de entrenamiento + prueba
distancia = np.random.uniform(1, 100, m) # km
peso = np.random.uniform(10, 1000, m) # kg
lluvia = np.random.binomial(1, 0.3, m) # 1 = llueve
hora_pico = np.random.binomial(1, 0.4, m) # 1 = sale en hora pico

# Regla "oculta" del mundo real (la red NO la conoce, debe aprenderla de los datos):
# la lluvia empeora más los viajes largos, y la hora pico solo afecta viajes de más de 40 km.
riesgo = (distancia/100)*(1 + 1.2*lluvia) + 0.6*hora_pico*(distancia > 40) \
        + 0.4*(peso/1000) + np.random.normal(0, 0.08, m)
tarde = (riesgo > 1.0).astype(int)

df = pd.DataFrame({"distancia_km": distancia, "peso_kg": peso,
                  "lluvia": lluvia, "hora_pico": hora_pico, "tarde": tarde})
display(df.head(8))
print(f"Porcentaje de pedidos tarde: {tarde.mean():.1%}")

fig, ax = plt.subplots(1, 3, figsize=(15, 4))

df.groupby("tarde")["distancia_km"].plot(kind="hist", alpha=0.5, bins=20, ax=ax[0],
legend=True)
ax[0].set_title("Distancia según el rótulo (0 = a tiempo, 1 = tarde)");
ax[0].set_xlabel("km")

(df.groupby("lluvia")["tarde"].mean()*100).plot(kind="bar", ax=ax[1],
color=["tab:blue", "tab:orange"])
ax[1].set_title("% de pedidos tarde según lluvia"); ax[1].set_xticklabels(["sin lluvia", "con lluvia"], rotation=0)

(df.groupby("hora_pico")["tarde"].mean()*100).plot(kind="bar", ax=ax[2],
color=["tab:blue", "tab:orange"])
```

```

ax[2].set_title("% de pedidos tarde según hora pico"); ax[2].set_xticklabels(["fuera de
pico", "hora pico"], rotation=0)
plt.tight_layout(); plt.show()

features = ["distancia_km", "peso_kg", "lluvia", "hora_pico"]
F = df[features].values
f_min, f_max = F.min(axis=0), F.max(axis=0)
Xn = (F - f_min) / (f_max - f_min)

idx = np.random.permutation(m)
n_train = int(0.8 * m)
tr, te = idx[:n_train], idx[n_train:]

X_train, Y_train = Xn[tr].T, df["tarde"].values[tr].reshape(1, -1)
X_test, Y_test = Xn[te].T, df["tarde"].values[te].reshape(1, -1)
print("X_train:", X_train.shape, "(features x ejemplos) | Y_train:", Y_train.shape)

def sig(z):
    # función de activación sigmoide
    return 1 / (1 + np.exp(-z))

def inicializar(topologia):
    # topologia = [n_entradas, neuronas capa 1, ...,
    neuronas de salida]
    W = [np.random.randn(topologia[i+1], topologia[i]) * 0.5 for i in
range(len(topologia)-1)]
    b = [np.zeros((topologia[i+1], 1)) for i in range(len(topologia)-1)]
    return W, b

def propaga(X, W, b):
    # Z(i) = W(i) A(i-1) + B(i); A(i) = sig(Z(i))
    A = [X]
    for Wi, bi in zip(W, b):
        A.append(sig(Wi @ A[-1] + bi))
    return A
    # A[-1] es la salida Hw(X)

def backpropagation(A, Y, W):
    m = Y.shape[1]
    L = len(W)
    e = Y - A[-1]
    delta = e * A[-1] * (1 - A[-1])
    # error: e = Yr - Y
    # delta de la última capa = diag(sig'(Z))
    e (sig' = sig(1-sig))
    dW, db = [None]*L, [None]*L
    for k in reversed(range(L)):
        dW[k] = delta @ A[k].T / m
        db[k] = delta.sum(axis=1, keepdims=True) / m
        # componente de corrección de W
        # componente de corrección del
sesgo B
        if k > 0:
            # se pasa el error hacia atrás: delta(k) =
diag(sig') W(k+1)^T delta(k+1)

```

```

        delta = (W[k].T @ delta) * A[k] * (1 - A[k])
    return dW, db

def entrenar(topologia, eta, epocas):
    W, b = inicializar(topologia)
    historia = {"perdida": [], "acc_train": [], "acc_test": []}
    for i in range(epocas):
        A = propaga(X_train, W, b)
        dW, db = backpropagation(A, Y_train, W)
        for k in range(len(W)):
            W[k] += eta * dW[k]
            b[k] += eta * db[k]
        if i % 50 == 0:
            e = Y_train - A[-1]
            historia["perdida"].append(0.5 * np.mean(e**2))
            historia["acc_train"].append(exactitud(X_train, Y_train, W, b))
            historia["acc_test"].append(exactitud(X_test, Y_test, W, b))
    return W, b, historia

def exactitud(X, Y, W, b):
    return ((propaga(X, W, b)[-1] > 0.5) == Y).mean()

W_a, b_a, h_a = entrenar([4, 1], eta=5, epocas=5000)
W_b, b_b, h_b = entrenar([4, 8, 4, 1], eta=5, epocas=5000)

print(f"Modelo A (1 neurona)      -> exactitud entrenamiento:
{exactitud(X_train,Y_train,W_a,b_a):.1%} | prueba:
{exactitud(X_test,Y_test,W_a,b_a):.1%}")
print(f"Modelo B (capas ocultas) -> exactitud entrenamiento:
{exactitud(X_train,Y_train,W_b,b_b):.1%} | prueba:
{exactitud(X_test,Y_test,W_b,b_b):.1%}")

epocas_eje = np.arange(len(h_b["perdida"])) * 50
fig, ax = plt.subplots(1, 2, figsize=(12, 4))
ax[0].plot(epocas_eje, h_a["perdida"], label="1 neurona")
ax[0].plot(epocas_eje, h_b["perdida"], label="Red con capas ocultas")
ax[0].set_title("Evolución de la función de pérdida E"); ax[0].set_xlabel("Épocas");
ax[0].legend(); ax[0].grid(True)
ax[1].plot(epocas_eje, h_a["acc_test"], label="1 neurona (prueba)")
ax[1].plot(epocas_eje, h_b["acc_test"], label="Red con capas ocultas (prueba)")
ax[1].set_title("Exactitud en el conjunto de prueba"); ax[1].set_xlabel("Épocas");
ax[1].legend(); ax[1].grid(True)
plt.tight_layout(); plt.show()

pred = (propaga(X_test, W_b, b_b)[-1] > 0.5).astype(int).flatten()
real = Y_test.flatten()

```

```

tp = int(((pred==1)&(real==1)).sum()); tn = int(((pred==0)&(real==0)).sum())
fp = int(((pred==1)&(real==0)).sum()); fn = int(((pred==0)&(real==1)).sum())

print(pd.DataFrame([[tn, fp], [fn, tp]],
                    index=["Real: a tiempo", "Real: tarde"],
                    columns=["Predicho: a tiempo", "Predicho: tarde"])))
print(f"\nExactitud: {(tp+tn)/len(real):.1%}")
print(f"Precisión (de los que dijo 'tarde', cuántos lo eran): {tp/(tp+fp):.1%}")
print(f"Recall (de los que realmente fueron tarde, cuántos detectó): {tp/(tp+fn):.1%}")

def predecir_pedido(distancia_km, peso_kg, lluvia, hora_pico):
    x = (np.array([distancia_km, peso_kg, lluvia, hora_pico], dtype=float) - f_min) /
    (f_max - f_min)
    p = propaga(x.reshape(-1, 1), W_b, b_b)[-1].item()
    veredicto = "LLEGARÁ TARDE" if p > 0.5 else "llegará a tiempo"
    print(f"{distancia_km} km, {peso_kg} kg, lluvia={lluvia}, hora pico={hora_pico} ->
    prob. de retraso = {p:.0%} -> {veredicto}")

predecir_pedido(80, 500, 0, 0)
predecir_pedido(80, 500, 1, 0) # mismo pedido, ahora con lluvia
predecir_pedido(60, 300, 0, 1) # viaje medio en hora pico
predecir_pedido(10, 800, 1, 1) # viaje corto

```

Explicación del código

El programa comienza generando un historial simulado de 3000 pedidos mediante NumPy. Para cada pedido se asignan valores aleatorios de distancia y peso, además de variables binarias para indicar si había lluvia o si el envío se realizaba en hora pico. A partir de estas variables se construye una condición de riesgo que determina si el pedido será clasificado como tardío. La idea es simular un problema real en el que diferentes factores de operación afectan el tiempo de entrega.

Después, el código analiza las características y el rótulo mediante gráficas. Se observa cómo cambia la cantidad de pedidos tardíos dependiendo de la distancia, la lluvia y la hora pico. Esta etapa permite identificar relaciones entre las variables de entrada y el resultado que la red debe aprender, siguiendo el concepto de features y rótulos utilizado en el documento.

Posteriormente, las cuatro características se normalizan al intervalo de 0 a 1 para que tengan escalas comparables. Los datos se dividen aleatoriamente en un 80 % para entrenamiento y 20 % para prueba. De esta manera, la red aprende utilizando una parte de los pedidos y posteriormente se comprueba su desempeño con datos que no utilizó durante el entrenamiento, lo cual permite evaluar su capacidad de generalización.

La red neuronal se construye directamente con NumPy, sin utilizar una librería especializada para el entrenamiento. Primero se define la función sigmoide y luego se crean funciones para inicializar los pesos y sesgos, realizar la propagación hacia adelante y ejecutar el algoritmo de Backpropagation. En la propagación, las entradas pasan por cada capa mediante una combinación de pesos y sesgos y posteriormente se aplica la función de activación. Esto sigue las ecuaciones de propagación descritas en el documento.

Una parte importante del ejercicio consiste en comparar una sola neurona con una red de dos capas ocultas. El primer modelo utiliza la topología [4,1], mientras que el segundo utiliza [4,8,4,1]. El código entrena ambos modelos durante 5000 épocas utilizando descenso del gradiente y retropropagación. La comparación permite observar cómo las capas ocultas aumentan la capacidad de la red para representar relaciones más complejas, aspecto relacionado con las limitaciones de una sola neurona mostradas en el documento.

Los resultados muestran que la neurona individual alcanza aproximadamente 90,8 % de exactitud en entrenamiento y 91,5 % en prueba, mientras que la red con capas ocultas alcanza 95,9 % en entrenamiento y 96,7 % en prueba. Esto evidencia que la segunda arquitectura logra aprender mejor las relaciones entre las variables, especialmente aquellas que dependen de la combinación de factores como distancia, lluvia y hora pico. El proceso de aprendizaje se realiza calculando el error, propagándose hacia las capas anteriores y actualizando los pesos mediante una tasa de aprendizaje, tal como se explica en el algoritmo de Backpropagation.

Finalmente, el mejor modelo se evalúa mediante una matriz de confusión, obteniendo una exactitud de 96,7 %, una precisión de 96,3 % y un recall de 96,3 %. Además, el programa permite introducir un pedido nuevo y obtener una probabilidad de retraso. Por ejemplo, al comparar un mismo envío de 80 km y 500 kg con y sin lluvia, la probabilidad de retraso aumenta considerablemente cuando se incorpora la lluvia. Esto demuestra de manera práctica cómo la red utiliza los patrones aprendidos para realizar predicciones sobre nuevos casos.

Explicación de los resultados obtenidos

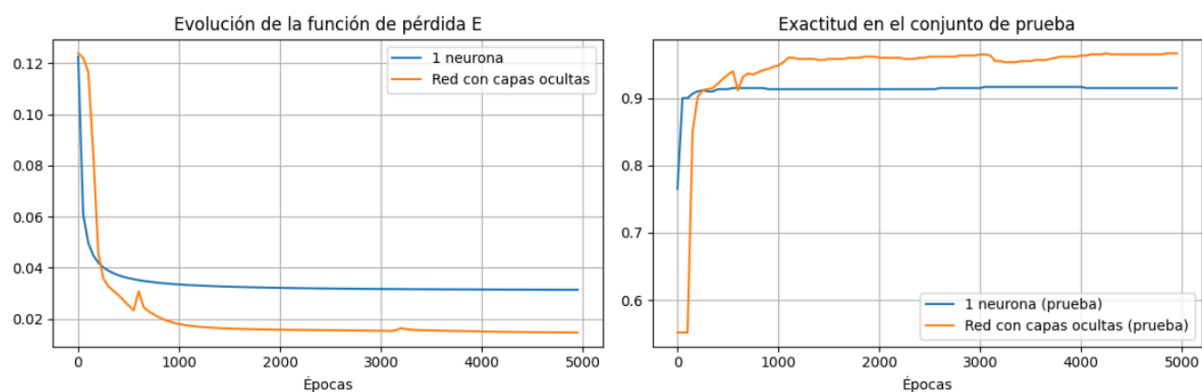
	distancia_km	peso_kg	lluvia	hora_pico	tarde
0	8.554521	693.120455	0	0	0
1	78.211960	676.233355	0	0	1
2	44.402514	898.277194	0	0	0
3	72.623053	348.044744	0	0	0
4	97.820962	551.705395	1	0	1
5	54.311091	221.461330	1	1	1
6	50.610926	339.478243	1	0	1
7	8.133062	823.631842	0	0	0

Porcentaje de pedidos tarde: 44.3%

El conjunto de datos utilizado contiene 3000 pedidos, de los cuales aproximadamente el 44,3 % fueron clasificados como tardíos. Las variables consideradas fueron la distancia, el peso, la presencia de lluvia y la hora pico. Los gráficos muestran que los pedidos de mayor distancia presentan una mayor frecuencia de retrasos y que tanto la lluvia como la hora pico incrementan notablemente el porcentaje de entregas tardías. Esto confirma que las variables seleccionadas tienen una relación importante con el comportamiento que la red debe aprender.

Modelo A (1 neurona) -> exactitud entrenamiento: 90.8% | prueba: 91.5%
 Modelo B (capas ocultas) -> exactitud entrenamiento: 95.9% | prueba: 96.7%

Se compararon dos modelos: una neurona individual y una red con dos capas ocultas. El primer modelo alcanzó una exactitud de 90,8 % en entrenamiento y 91,5 % en prueba, mientras que el segundo alcanzó 95,9 % y 96,7 %, respectivamente. Esto demuestra que la incorporación de capas ocultas permite representar mejor la relación entre las variables de entrada y el retraso de los pedidos.



La evolución de la función de pérdida muestra que ambos modelos aprenden rápidamente durante las primeras épocas, pero la red con capas ocultas logra reducir el error a un valor mucho menor, cercano a 0,015, frente a aproximadamente 0,031 de la neurona individual. De forma similar, la exactitud del modelo con capas ocultas alcanza aproximadamente 96,7 %, superando claramente al modelo de una sola neurona.

	Predicho: a tiempo	Predicho: tarde
Real: a tiempo	321	10
Real: tarde	10	259

Exactitud: 96.7%
 Precisión (de los que dijo 'tarde', cuántos lo eran): 96.3%
 Recall (de los que realmente fueron tarde, cuántos detectó): 96.3%

La matriz de confusión del modelo seleccionado muestra 321 casos correctamente identificados como pedidos a tiempo y 259 como tardíos, con pocos errores de clasificación. Esto se refleja en una exactitud de 96,7 %, una precisión de 96,3 % y un recall de 96,3 %,

evidenciando un desempeño equilibrado tanto para detectar pedidos tardíos como para evitar falsas predicciones.

```
80 km, 500 kg, lluvia=0, hora pico=0 -> prob. de retraso = 57% -> LLEGARÁ TARDE
80 km, 500 kg, lluvia=1, hora pico=0 -> prob. de retraso = 100% -> LLEGARÁ TARDE
60 km, 300 kg, lluvia=0, hora pico=1 -> prob. de retraso = 100% -> LLEGARÁ TARDE
10 km, 800 kg, lluvia=1, hora pico=1 -> prob. de retraso = 1% -> llegará a tiempo
```

Finalmente, las pruebas con nuevos pedidos muestran que el modelo responde de acuerdo con las condiciones de entrada. Por ejemplo, un pedido de 80 km y 500 kg pasa de una probabilidad de retraso del 57 % sin lluvia a 100 % con lluvia. También se observa que combinar lluvia y hora pico aumenta considerablemente la probabilidad de retraso, mientras que un pedido más corto, fuera de hora pico y sin lluvia presenta una probabilidad cercana al 1 %. Esto demuestra que la red aprendió las relaciones entre los factores logísticos y el resultado esperado.